



UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR

Cahier de Réalisation : DISK CLONER

Présenté par :

- HARVIS LANDRY FOTSEU FONKWA
- TCHINWA ISMAEL CLINTON

Étudiants à Saint Jean Ingénieur



Sous la supervision de :

- M. Emmanuel NGUIMBUS,

Enseignant de SE à Saint Jean Ingénieur

PLAN



01	INTRODUCTION
02	ANALYSE DES BESOINS
03	CHOIX DES TECHNOLOGIES
04	ARCHITECTURE TECHNIQUE
05	PHASE DE REALISATION
06	CONSTRAINTES RENCONTRÉES
07	CONCLUSION ET PERSPECTIVES



01

INTRODUCTION



CONTEXTE

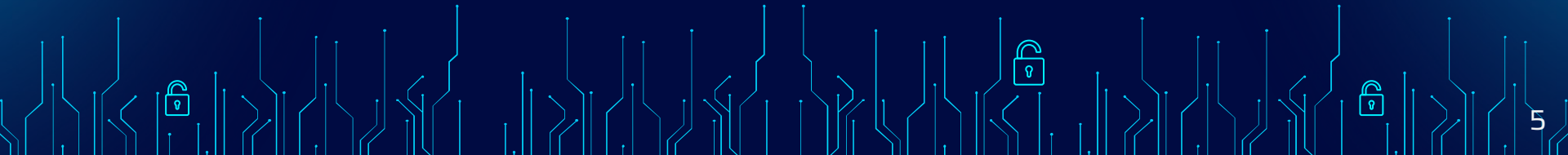
Dans les environnements professionnels et académiques, la migration et la sauvegarde des données constituent des opérations critiques nécessitant fiabilité et précision.

OpenSolaris (OpenIndiana) est un système d'exploitation orienté stockage, réputé pour son système de fichiers ZFS et sa gestion avancée des volumes. Cependant, les outils de clonage disponibles restent souvent limités à des commandes bas niveau complexes.

L'outil standard dd, bien que puissant, ne propose aucune interface visuelle ni retour en temps réel sur la progression des opérations — ce qui représente un risque opérationnel important.

PROBLÉMATIQUE

Comment concevoir un système de clonage de disque bas niveau sous OpenSolaris, offrant la puissance de dd avec une interface graphique moderne permettant le suivi en temps réel des opérations ?



OBJECTIFS

- Comprendre la structure bas niveau d'un disque Solaris
- Réimplémenter dd en C avec gestion des erreurs I/O
- Développer une interface web moderne (Angular + Flask)
- Garantir la sécurité des opérations destructives



02

ANALYSE DES BESOINS



BESOINS FONCTIONNELS

- **Inventaire automatique:** Le système doit scanner le bus de données et lister les disques physiques et leurs capacités, les systèmes de fichier.
- **Création d'image disque:** Permettre la lecture d'un disque source et son écriture dans un fichier .img compressé pour l'archivage.
- **Restauration d'image :** Permettre de réinjecter un fichier image vers un disque physique cible.
- **Clonage direct (Disk-to-Disk) :** Transférer les données d'un disque A vers un disque B sans stockage intermédiaire.
- **Monitoring en temps réel :** Afficher graphiquement le débit (Mo/s), le volume déjà copié et le pourcentage de complétion.
- **Gestion des Logs :** Enregistrer chaque événement (succès, erreur de lecture, interruption) dans un fichier de journalisation.

BESOINS NON FONCTIONNELS

- **Performance :** Optimisation de la taille des blocs de lecture (buffer) pour minimiser les accès aux têtes de lecture.
- **Sécurité:** Imposer un verrouillage logiciel sur le disque monté sur / (racine) pour empêcher son altération accidentelle.
- **Ergonomie :** Interface intuitive permettant de réaliser un clonage en moins de 5 clics après sélection des unités.
- **Fiabilité :** Utilisation d'un mode de lecture "Raw" (périphérique en mode caractère) pour garantir l'exactitude des données.
- **Simplicité d'utilisation**

03

CHOIX DES TECHNOLOGIES



CHOIX DES TECHNOLOGIES

Justification de la stack

technique



TECHNOLOGIES	JUSTIFICATION	ALTERNATIVES
C (GCC 13) Moteur de clonage bas niveau	Accès direct aux syscalls Solaris (open, read, write, ioctl), manipulation des raw devices /dev/rdsk/, contrôle précis des buffers et gestion des signaux.	Alt. écartée: Python pur) Trop lent pour les I/O intensifs
Python + ctypes Couche d'intégration	ctypes permet d'appeler libclone.so depuis Python sans recompilation. Facilite l'intégration avec Flask et le threading pour le polling de statut.	Alt. écartée: JNI (Java)) Complexité de configuration excessive
Flask (Python) API REST Backend	Léger, facile à déployer sur OpenIndiana, excellent support CORS, idéal pour exposer les fonctions C via HTTP avec validation des paramètres.	Alt. écartée: Django / FastAPI - Surcharge pour un petit projet
Angular 17 Frontend SPA	Framework robuste avec routing, services HTTP, RxJS pour le polling temps réel et TypeScript pour la sécurité des types. Architecture composants claire.	Alt. écartée: React / Vue.js - Moins structuré pour les grandes apps
Bootstrap 5 UI / Responsive	Composants prêts à l'emploi (badges, progress bars, tables, modals), thème dark natif, compatibilité complète avec Angular.	Alt. écartée: Tailwind CSS - Nécessite un compilateur en dev
OpenIndiana Système d'exploitation	Héritier direct d'OpenSolaris, maintenu activement. Fournit ZFS, /dev/rdsk/, VTOC, DKIOCGMEDIAINFO - toutes les APIs Solaris nécessaires au projet.	Linux / FreeBSD - Ne dispose pas des APIs Solaris requises



04

ARCHITECTURE TECHNIQUE



ARCHITECTURE TECHNIQUE

Architecture 3-couches - Moteur C / API Flask / Frontend Angular

COUCHE PRÉSENTATION - Angular 17 + Bootstrap 5

Dashboard
(liste disques)

Clone Wizard
(4 étapes)

Progress Monitor

Log Viewer

Navbar

HTTP REST / JSON

COUCHE MÉTIER - Flask REST API + Python ctypes

app.py
(endpoints REST)

engine_wrapper.py
(ctypes bridge)

Validation
(sécurité)

Threading
(async clone)

CORS
(Angular - Flask)

ctypes / libclone.so

COUCHE SYSTÈME - Moteur C (libclone.so) - OpenSolaris Raw I/O

clone_engine.c
(boucle read/write)

disk_manager
(list_disks)

security.c
(mnttab/rpool)

progress.c
(callback UI)

logger.c
(/var/log)

05

PHASE DE REALISATION



PHASE DE RÉALISATION Réimplémentation de dd - lecture/écriture bloc par bloc

clone_engine.c - Boucle principale

```
int clone_disk(CloneParams *p) {
    // Ouverture raw devices Solaris
    int fd_src = open(p->src, O_RDONLY);
    int fd_dst = open(p->dst, O_WRONLY|O_CREAT, 0644);

    // Taille via ioctl Solaris
    ioctl(fd_src, DKIOCGMEDIAINFO, &info);
    uint64_t total = info.dki_capacity * info.dki_lbsize;

    // Boucle de copie bloc par bloc
    while ((n = read(fd_src, buf, bs)) > 0) {
        write(fd_dst, buf, n);
        done += n;
        // Callback vers UI (Flask polling)
        progress_cb(done, total);
    }

    fsync(fd_dst); // Flush physique
    return ERR_OK;
}
```

POINTS CLÉS

Raw Device Access

Ouverture directe de /dev/rdisk/ sans passer par le VFS - accès bit à bit identique à dd

ioctl DKIOCGMEDIAINFO

API Solaris pour obtenir la taille exacte du disque en secteurs * taille lbsize

Buffer 1 Mo

Optimisation du débit : 1 Mo par cycle atteint 50-60 Mo/s sur disques virtuels VirtualBox

Callback progression

Pointeur de fonction passé au moteur, appelé après chaque bloc pour mise à jour de l'UI

fsync() obligatoire

Garantit que les données sont physiquement écrites sur le disque avant fermeture

Sécurité mnttab

Vérification de /etc/mnttab avant toute écriture pour éviter la corruption

RÉALISATION - RÉSULTATS ET TESTS Validation de toutes les fonctionnalités requises

8 / 8 tests validés

T1 Inventaire automatique

5 disques détectés avec taille, filesystem ZFS/UFS, état monté/libre et protection système

T2 Clonage Disk → Image

c2d1p0 (4.0 GB) → /export/images/backup.img en 1:09 à 51.7 Mo/s — 100% complété

T3 Clonage Image → Disk

backup.img restauré sur c2d1p0 — fichiers test1.txt, test2.txt, app.py vérifiés identiques

T4 Clonage Disk → Disk

c2d1p0 → c3d1p0 (4.0 GB) en 1:09 à 58.9 Mo/s - données vérifiées avec cat après montage

T5 Sécurité - Disque système

c2d0p0 (rpool) détecté via zpool status - badge SYSTÈME rouge, bouton Cloner désactivé

T6 Sécurité - Confirmation

Saisie CLONER obligatoire - toute autre saisie bloque le bouton Lancer le clonage

T7 Monitoring temps réel

Barre progression + Mo/s + temps écoulé mis à jour chaque seconde via polling Flask

T7 Journalisation

Tous les événements dans /var/log/diskcloner.log avec horodatage [INFO]/[ERROR]

06

CONTRAINTES RENCONTRÉES



CONTRAINTES RENCONTRÉES

Difficultés rencontrées et solutions apportées

Difficultés	solutions apportées
Réseau DNS dans la VM pkg.openindiana.org ne résolvait pas malgré une connexion internet fonctionnelle. Les paquets ne pouvaient pas être téléchargés.	Ajout manuel de l'IP dans /etc/hosts (65.21.23.2 pkg.openindiana.org) et configuration des DNS (8.8.8.8, 9.9.9.9) dans /etc/resolv.conf.
Format des devices Solaris: La première version du code cherchait cotodo (avec t) comme indiqué dans la documentation mais OpenIndiana sur VirtualBox utilise c2dop0 (sans t). Aucun disque n'était détecté.	Modification du grep dans list_disks() pour supporter les deux formats et utiliser po (partition entière).
Permissions et root: Flask installé pour le simple utilisateur mais pas pour root. Le backend en root ne trouvait pas les modules flask/flask-cors.	Réinstallation de Flask en root
Création de fichiers image: open() avec O_WRONLY échouait si le fichier n'existait pas. Error 0 retourné sans message d'erreur explicite.	Ajout de O_CREAT O_TRUNC aux flags d'ouverture de la destination, avec permissions 0644 comme paramètre mode.
Détection filesystem: fstyp retournait ZFS pour tous les disques car le pool ZFS principal couvre le système. UFS sur c2d1 n'était pas reconnu.	Lecture directe de /etc/mnttab pour récupérer le vrai type de filesystem associé à chaque device via getmntent().

07

CONCLUSION ET PERSPECTIVES



BILAN DU PROJET

★ Objectifs atteints

- Toutes les fonctionnalités requises sont implémentées et testées avec succès sur OpenIndiana réel.

★ Compréhension bas niveau

- Le moteur C réécrit from scratch démontre la maîtrise des raw devices, ioctl Solaris et des opérations I/O bloc par bloc.

★ Stack moderne validée

- L'architecture C → ctypes → Flask → Angular prouve la faisabilité d'une interface web moderne pour des outils système bas niveau.

COMPÉTENCES ACQUISES

- ❖ Structure physique d'un disque Solaris (slices, VTOC, ZFS pools)
- ❖ Manipulation des raw devices `/dev/rdisk/` avec `open/read/write`
- ❖ Appels système `ioctl` pour la géométrie des disques
- ❖ Création d'une bibliothèque partagée `.so` en C
- ❖ Bridge Python/C avec `ctypes`
- ❖ API REST Flask avec `threading` asynchrone
- ❖ Composants Angular avec `polling RxJS` temps réel

PERSPECTIVES D'ÉVOLUTION

➤ Compression ZFS

- Support natif des snapshots ZFS send/receive pour des images compressées et incrémentales

➤ Vérification MD5

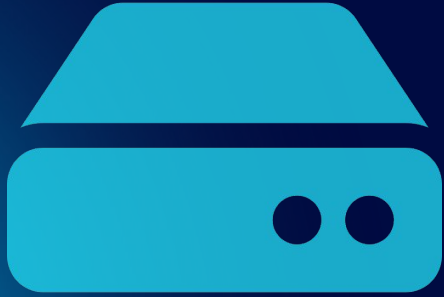
- Hash de la source et de la destination après clonage pour garantir l'intégrité bit à bit

➤ Clonage réseau

- Transfert via SSH/netcat vers un serveur distant pour les sauvegardes hors site

➤ Planificateur

- Interface de scheduling pour les sauvegardes automatiques périodiques



Merci pour votre attention!

Des questions?

